

BackupFabric

An independent Linux control plane for isolated backup operations

Receiving, discovering, comparing, and retaining backup histories from multiple sources, with explicit evidence and controlled recovery workflows.

Audience

Platform engineers, infrastructure operators, and security architects

Publication date

8 October 2026

Research and experimental implementation

Independent project. BackupFabric is not affiliated with, sponsored by, endorsed by, or an official product of GitHub, Inc. No vendor certification or support commitment is implied.

GitHub Enterprise Server is discussed as the initial backup producer evaluated in the laboratory. References to vendor documentation identify source behavior; they do not establish support for BackupFabric's experimental receiver topology.

This paper describes the implementation and evidence available on the publication date. BackupFabric remains experimental and is not production-ready.

01 | EXECUTIVE SUMMARY

Make backup operations observable. Do not confuse copies with recovery.

BackupFabric separates the operational work of managing backup histories from the application that creates and restores them.

The initial implementation focuses on GitHub Enterprise Server (GHES). GHES remains responsible for producing its native backup format and executing native restoration. BackupFabric provides a Linux-based registry, receiver inventory, operation journal, verification workflow, retained-history copies, and management interface.

The problem is not simply transporting files. An operator must know which source a history belongs to, whether a publication is complete, what a comparison establishes, whether a retained copy exists, and what recovery has actually demonstrated. Combining these questions into a single green status creates false confidence.

Three operating principles

1. **Source isolation.** Credential-to-receiver bindings determine storage access. Source-provided timestamps and paths do not select another source's namespace.
2. **Evidence before promotion.** Discovery, comparison, retention, independent protection, and restore qualification are distinct facts. None substitutes for all the others.
3. **Explicit operational authority.** Managed operations require approval. Verification and retention require coordinated writers. Restore execution remains manual.

What the laboratory establishes

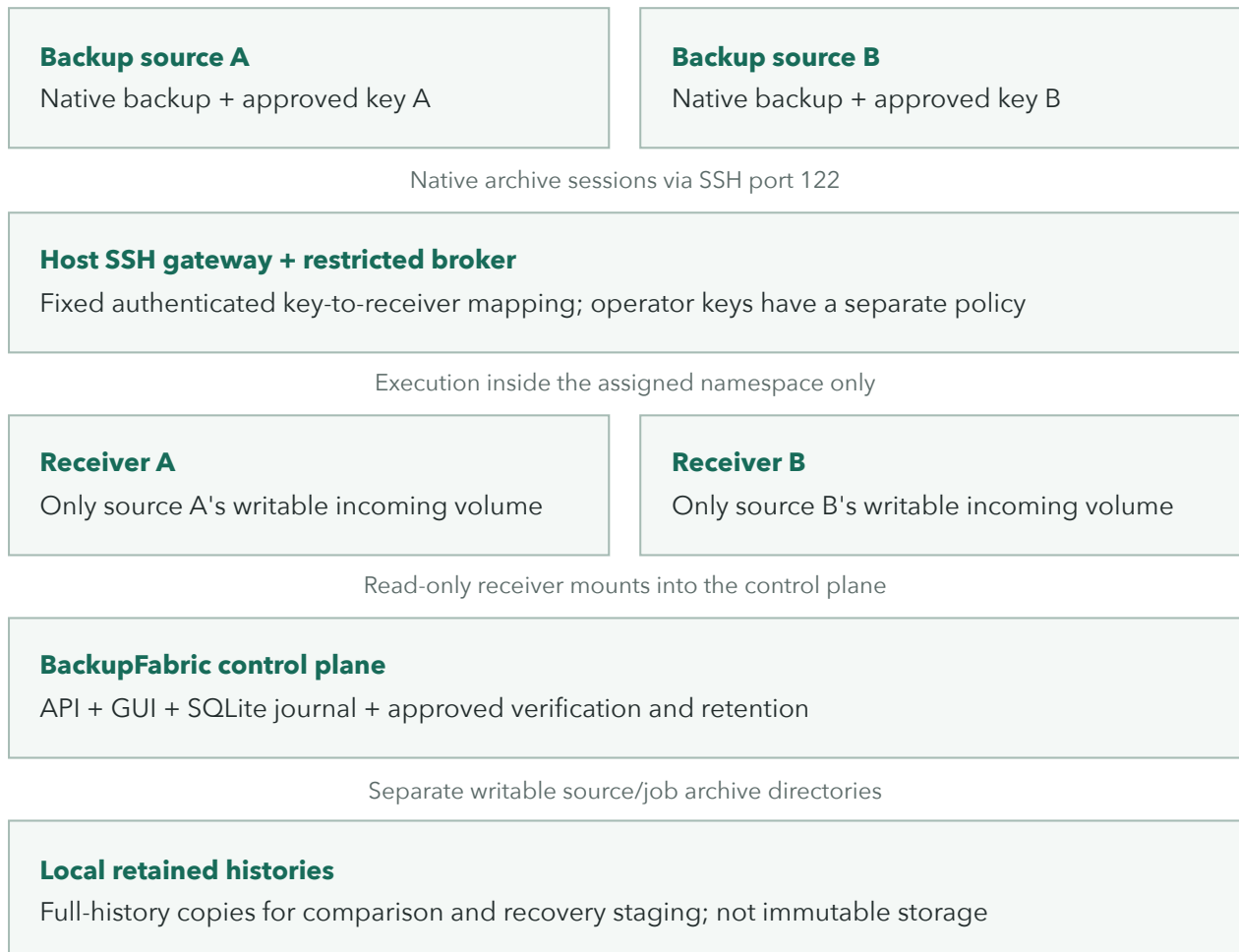
Two GHES sources on releases 3.21.6 and 3.21.7 completed controlled backup, delivery, resend, comparison, and retention pipelines. A later GHES 3.21.7 data restore consumed a BackupFabric-retained history and passed repository and service checks.

Interpretation: this is bounded laboratory evidence, not a general compatibility guarantee, independently protected disaster recovery, or production certification. Broader feature validation and clean-host deployment qualification remain outstanding.

The intended value is a clearer, repeatable backup operating model with attributable source histories, visible retention gaps, and a disciplined path toward demonstrated recovery.

02 | ARCHITECTURE

One entry point. Separate histories.



Container roles and authority

The current native-push prototype uses Docker Engine for per-source receiver isolation. Each receiver has a read-only root, disabled networking, dropped capabilities, a configured non-root identity, and one writable backup volume. The host broker invokes commands inside the selected container.

The control plane does not receive the Docker socket or host broker privileges. It reads receiver data through read-only mounts and writes its catalog and retained copies to separate paths. The Go control plane can run natively; a container-free native-push isolation alternative is not implemented.

The documented GHES remote-archive destination is another GHES appliance. BackupFabric's Ubuntu receiver namespaces are an experimental compatibility path outside that documented topology.

03 | WORKFLOW

A journal that describes what happened, not what was assumed.

Operation	Behavior and evidence boundary
Create backup and receive	Requests a native source backup, awaits archive completion, and checks that receiver publication matches source metadata. Does not infer content integrity or restore qualification.
Receive existing backup	Sends source current to the assigned receiver. Does not create a new source snapshot; an identical resend may reuse the same timestamp.
Discover snapshots	Validates bounded timestamp/version/UUID metadata and publication markers. Records one durable observation per source/timestamp, not a witnessed transfer event.
Verify received data	Compares source and receiver checksums and archive metadata with stable publication checks. Requires paused writers and does not prove source cleanliness.
Retain complete history	Copies a coordinated history to a fresh source/job directory, preserving required metadata and comparing the result. Failure does not publish a usable collection.
Prepare restore target	Copies retained history to a separately registered compatible GHES target with an empty backup root. Does not run native restore or format disks.

Publication and retention detection

Native metadata reads are bounded and reject malformed versions, wrong source identities, symlink/FIFO metadata, and incomplete publications. Reconciliation occurs at startup, GUI refresh, native preflight, and operation pre-checks; an API/CLI scan is also available.

Repeated and concurrent scans are idempotent. An observation's timestamp records discovery, not transfer completion. Separate resends of one unchanged snapshot are not individually audited. There is no independent always-on filesystem watcher.

The inventory distinguishes pending collection from retained, unqualified histories using completed collection evidence and matching archived metadata. A full-history collection can include several native timestamps and versions.

Writer quiescence is still an operator attestation. Stable metadata or an absent marker alone cannot coordinate every independent backup, prune, or transfer writer.

04 | SECURITY MODEL

Isolation is necessary. Immutability is a different property.

A compromised source must not obtain authority over another source. It must also not be mistaken for a trusted producer simply because its backup can be copied accurately.

Approved incoming keys select fixed receiver namespaces. Tested source sessions could not access host data, control credentials, host sudo, the Docker socket, or another receiver root. Forwarding to the management API was denied for backup-source keys.

These controls have a specific limit: the broker permits shell execution inside the assigned receiver, whose incoming volume is writable. A stolen source key can therefore damage that source's own incoming history. No broader container-escape or adversarial security certification has been completed.

Storage tier	Current protection and limitation
Incoming history	Per-source namespace isolation. Writable by the authorized source identity; not immutable.
Local retained history	Not mounted into source receiver sessions. Writable by the control plane and within the same host/storage failure boundary.
Independent recovery store	Proposed only. No retention-lock backend, separated storage authority, or independently protected-success state is implemented.

Management access

The current API is unauthenticated and restricted to loopback. Operators access it through a trusted SSH tunnel on the existing port 122. Host/origin/JSON guards reduce browser request risks but do not replace authentication, authorization, or trusted-operator controls.

Design rules

- Keep source backup keys distinct from reverse-control credentials and operator keys.
- Pin source SSH identities; do not enroll unknown keys automatically.
- Do not relax ownership, link, ACL, or extended-attribute checks to make a failed transfer appear successful.
- Retain earlier recovery evidence when new incoming content is suspect. Checksums demonstrate copy fidelity, not malware detection.

05 | DEPLOYMENT

Ubuntu-first. Installation inputs, not lab identities.

Ubuntu 24.04 LTS is the recommended host baseline. The project aims for broader Linux deployment, but distribution, architecture, filesystem, kernel, service-manager, and security-policy combinations require qualification.

Current deployment requirements

- Docker Engine for receiver isolation; Compose for the control-plane quick start.
- Persistent, adequately sized storage with the required link and metadata support.
- Approved identities, pinned keys, isolated routes, compatible ownership, and trusted operator SSH access.

Runtime assets contain no operational keys or real appliance bindings. Generic configurations and external fixtures supply installation inputs. Image build arguments and fixed broker routes configure UID/GID; unsupported ownership fails closed.

Release and onboarding work remains

The repository quick start launches the control plane, not a fully provisioned backup service. Receiver setup and persistent mounts remain separate. A guided installer, released multi-architecture packages, automatic trusted onboarding, and broader Linux certification are planned.

Version handling

Versions are recorded per snapshot; upgrades do not rewrite earlier version evidence. Managed checks match source/receiver metadata. Version parsing is not format certification, and restore compatibility still requires operator confirmation.

Beyond GHES

A generic adapter could catalog LAMP database dumps and application files using approved source bindings, isolated namespaces, bounded manifests, checksums, and explicit completion. Producers remain responsible for consistent backups and application-specific restoration.

Generic ingestion is proposed, not implemented. Arbitrary files pushed to a shared path must not be treated as completed backups or disguised with fake GHES metadata.

06 | DEMONSTRATED BEHAVIOR

Live transfers. A bounded restore rehearsal.

Test area	Recorded outcome
Two-source pipelines	GHER 3.21.6 and 3.21.7 each completed fresh backup, archive delivery, explicit resend, comparison, and full-history retention: eight real managed operations.
Isolation	Actual source-key host-path and forwarding denials; harmless same-path fixtures remained separate. Portable broker tests also exercised concurrent writes and links.
Integrity and history	Newest snapshots passed checksum/archive-metadata comparisons, including dry-run extra-entry checks. Older entries were recognized inside retained full histories.
Durability of records	Controlled service restart preserved 22 journal records, settings, retained-current references, and idempotent discovery for the recorded run.
Regression checks	Linux tests covered API guards, failed/changed transfers, metadata safety, ownership constraints, concurrency, retention lookup, and fixture-based restore staging.

Retained-history restore

A separate approved rehearsal copied a retained source history back to its original GHER 3.21.7 appliance. The original local backup tree was preserved. Staging used metadata-preserving copy and comparison before manual native restoration without the configuration flag.

The native restore exited successfully. Its reported runtime was 1,580 seconds, approximately 26 minutes 20 seconds. This excludes portions of preparation and subsequent validation and is not a recovery-time objective or performance benchmark.

Twenty-one repositories matched the normalized pre-restore reference digest and passed object integrity checks. Restore markers cleared, services were healthy, maintenance was removed, and local HTTP/API checks succeeded. BackupFabric was restarted afterward.

This was a same-appliance data restore, not a fresh-host disaster-recovery exercise. Authenticated users/organizations, external Actions data, runner recovery, and cross-version/cross-appliance restoration were not fully validated. No automatic restore-qualified catalog state was assigned.

07 | QUALIFICATION AGENDA

What remains before production adoption.

- **Independent protection:** select and validate a recovery store with separate writer, retention-administrator, and recovery-reader authority.
- **Recovery breadth:** execute seeded fresh-target rehearsals, preserve database dependencies, measure complete recovery time/point, and validate authenticated application behavior.
- **Operational resilience:** prove persistent mounts, boot ordering, host-loss recovery, catalog reconstruction, interrupt/retry handling, and realistic concurrent-source workloads.
- **Security readiness:** complete image and broker review, least-privilege administration, API authentication/authorization, and adversarial testing. An earlier base-image diagnostic remains unresolved.
- **Deployment and formats:** deliver safe initialization/release packaging, qualify Linux architectures/distributions, and implement generic producer adapters without weakening native validation.

Platform snapshots and native-push collection remain complementary. The current provider adapter generates development references only; it does not capture real storage snapshots. Filesystem-compatible replication, independent retention, and a successful restore are separate obligations.

Public references

1. BackupFabric source and technical documentation.
github.com/appataalks/BackupFabric
2. GitHub documentation: Configuring remote archives for backups.
[GHES 3.21 remote archive documentation](#)
3. GitHub documentation: Understanding the snapshot file structure.
[GHES 3.21 snapshot structure](#)
4. GitHub documentation: Restoring from a backup.
[GHES 3.21 restore procedure](#)
5. GitHub documentation: Restoring with GitHub Actions enabled.
[External Actions storage and recovery prerequisites](#)

Public documentation was consulted for producer behavior. External Actions logs, artifacts, and blobs require separate coverage. Repository lab records underpin this paper's test claims; they are not an independent audit or vendor certification.

Legal and attribution: BackupFabric is an independent Apache-2.0 project. GitHub and GitHub Enterprise Server are trademarks of their respective owner. Mention of those products does not imply affiliation, sponsorship, endorsement, or an official support relationship.